

Intro to GenAI — Hands-On API Lab (Python / Flask)

A progressive lab that takes you from a plain `/ping` route to a stateful, multi-turn chatbot — using the **OpenAI** and **Anthropic (Claude)** APIs from a single small **Flask** application.

The lab is split into two folders:

- **todo/** — the learner version: implement each exercise one by one.
- **solution/** — the completed version to compare against.

This is the Python port of the original Node.js "Intro to GenAI" lab. Same exercises, same routes — Flask instead of Express, the `openai` and `anthropic` Python SDKs instead of the JavaScript ones.

What you will learn

#	Exercise	Route	What it teaches
1	Ping / Pong	GET <code>/ping</code>	Wire up and test a route before adding any AI
2	Token counting	GET <code>/tokens</code>	What tokens are; count them with no API call
3	First GPT call	GET <code>/ask</code>	Call the OpenAI Chat Completions API
4	Temperature	GET <code>/temperature</code>	How randomness is controlled
4b	Reasoning model	GET <code>/gpt/reasoning</code>	Reasoning models + the Responses API (<code>effort</code>)
5	First Claude call	GET <code>/claude</code>	Call the Anthropic API; how it differs from OpenAI
6	GPT HTML page	GET <code>/gpt/html</code>	System messages + structured (HTML) output
7	Claude HTML page	GET <code>/claude/html</code>	Claude's top-level <code>system</code> field
8	Image generation	GET <code>/gpt/image</code>	Generate an image with <code>gpt-image-2</code> (base64)
9	POST route	POST <code>/gpt/ask</code>	Why real AI APIs use POST + a JSON body
10	Multi-turn chat	POST <code>/gpt/chat</code>	How chatbots work — the model is stateless

Prerequisites

- **Python** (latest) — verify with `python --version` (or `python3 --version` / `py --version` if `python` doesn't work).
- **An OpenAI API key** — <https://platform.openai.com/api-keys>
- **An Anthropic API key** — <https://console.anthropic.com/>

- A REST client for the POST exercises (9 and 10): **Postman**, **curl**, or the VS Code *REST Client / Thunder Client* extension.

Setup (once, before you start)

This lab uses **one virtual environment** shared by the whole workshop, created **once** at the **repo root**. We set it up here — it's the same environment the **tutorial/** part uses later, so you only do this one time. The packages (**Flask**, **openai**, **anthropic**, **tiktoken**, **python-dotenv**) are all in the root **requirements.txt**.

1. Create and activate the shared environment

From the **repo root** (the folder above **GenAI_Intro/**):

```
python -m venv venv          # skip this line if the venv folder already
exists

source venv/bin/activate     # macOS / Linux
venv\Scripts\activate        # Windows

pip install -r requirements.txt
```

After activating, your prompt shows **(venv)**.

To leave a venv, type **deactivate** (same on macOS / Linux / Windows). Because this one environment covers the whole workshop, you don't switch between this lab and the tutorial — keep it active for both.

2. Choose a folder to work in

You can work inside either **todo/** or **solution/**:

```
cd GenAI_Intro/todo          # or: cd GenAI_Intro/solution
```

3. Add the **.env** file with your API keys

In the folder you're working in (**todo/** or **solution/**), create a file named **.env** and put the two keys I'll give you inside it:

```
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
```

I'll hand you the keys (or a ready-made **.env**) during the workshop — just drop the file next to **app.py**. The app reads it automatically on startup.

4. Run the server

```
python app.py
```

You should see **Server running on <http://localhost:3000>**. Leave it running; it auto-reloads when you save **app.py**.

Models: the model names live in constants at the top of **app.py** (**GPT_MODEL**, **REASONING_MODEL**, **CLAUDE_MODEL**, **IMAGE_MODEL**). If your API key can't access one of them, change the constant to a model you do have — e.g. set **REASONING_MODEL** to whichever OpenAI reasoning model you can use.

The exercises, step by step

Work through these in order inside **todo/app.py**. Each one adds a route. After saving, test it with the URL shown (just open it in your browser, unless it says POST). Compare against **solution/app.py** whenever you get stuck.

Exercise 1 — Ping / Pong (GET **/ping**) — already done for you

The simplest possible route: prove the server works before adding any AI. This one is already implemented in **todo/app.py** as your worked example:

```
@app.get("/ping")
def ping():
    return "pong"
```

Test: open <http://localhost:3000/ping> → you should see **pong**.

Exercise 2 — Token counting (GET **/tokens**)

Tokens are the units models use to read and generate text — roughly 4 characters each in English. Counting them before a request lets you estimate cost and check the prompt fits in the context window. **tiktoken** is OpenAI's own tokeniser, so the count is exact with **no API call**.

1. Read `request.args.get("prompt")`.
2. `enc = tiktoken.encoding_for_model("gpt-4o")`
3. `token_ids = enc.encode(prompt)`
4. Return `jsonify(prompt=prompt, token_count=len(token_ids))`

Test: `/tokens?prompt=Hello+world` → `{"prompt": "Hello world", "token_count": 2}`

Exercise 3 — First GPT call (GET **/ask**)

Send the prompt to GPT-4o and return its reply.

```
completion = openai.chat.completions.create(
    model=GPT_MODEL,
    messages=[{"role": "user", "content": prompt}],
)
reply = completion.choices[0].message.content
```

Bonus: print `completion.usage` to see input / output / total token counts.

Test: `/ask?prompt=What+is+the+capital+of+France`

Exercise 4 — Temperature (GET `/temperature`)

Temperature is a value from 0 to 2:

Value	Behaviour
0	Very deterministic — nearly the same answer every time
1	Balanced — the default
2	Highly creative — sometimes unexpected

Read `temp` carefully: `temp=0` is valid, so default to `1` only when the value is **missing** (`None`), not when it's falsy. Pass `temperature=temp` and `max_tokens=1000`.

Test: run `/temperature?prompt=Give+me+one+word&temp=0` three times (same answer?), then `&temp=2` three times (more varied?).

Key finding: temperature only works on standard models like `gpt-4o`. Reasoning models use `reasoning.effort` instead — that's Exercise 4b.

Exercise 4b — Reasoning model (GET `/gpt/reasoning`)

Reasoning models think internally before answering. You **cannot** set temperature — instead you set `reasoning.effort` (`low` / `medium` / `high`). They use a different API, the **Responses API**:

```
response = openai.responses.create(
    model=REASONING_MODEL,
    reasoning={"effort": effort, "summary": "auto"},
    input=[{"role": "user", "content": prompt}],
)
reply = response.output_text      # note: no .choices[0] here
```

The reasoning summary lives in `response.output` — find the item whose `type == "reasoning"`, then read `item.summary[0].text` (if present).

Heads-up: `"summary": "auto"` requires a **verified** OpenAI organization ([verify here](#)). If yours isn't verified, the request fails with a 400 on `reasoning.summary` — just drop the `"summary"` key (`reasoning={"effort": effort}`) and the model still answers; you only lose the summary. The solution catches this and falls back automatically.

Test: compare `effort=low` vs `effort=high` on a riddle.

Exercise 5 — First Claude call (GET `/claude`)

Anthropic's API differs from OpenAI in three ways:

- the method is `claude.messages.create()`
- `max_tokens` is **required**
- the reply lives at `message.content[0].text`

```
message = claude.messages.create(
    model=CLAUDE_MODEL,
    max_tokens=1024,
    messages=[{"role": "user", "content": prompt}],
)
reply = message.content[0].text
```

Test: `/claude?prompt=What+is+the+capital+of+France`

Exercise 6 — GPT HTML page (GET `/gpt/html`)

A **system message** sets the model's role for the whole conversation. Ask GPT-4o to act as a web developer and output a complete HTML page. Always run the output through `strip_code_fences()` (provided in `app.py`) — models sometimes wrap HTML in ````html` fences. Return it with `Response(html, mimetype="text/html")`.

Test: `/gpt/html?subject=the+solar+system` → a rendered page in the browser.

Exercise 7 — Claude HTML page (GET `/claude/html`)

Same task, with Claude. With Anthropic, `system` is a **top-level field**, not part of `messages[]`:

```
message = claude.messages.create(
    model=CLAUDE_MODEL,
    max_tokens=10000,
    system="You are an expert web developer. Output only valid, complete HTML — nothing else.",
    messages=[{"role": "user", "content": user_prompt}],
)
```

Test: open `/gpt/html?subject=X` and `/claude/html?subject=X` side by side and compare.

Exercise 8 — Image generation (GET `/gpt/image`)

`openai.images.generate()` with the `gpt-image-2` model returns the image as **base64 data** (`data[0].b64_json`), **not** a URL:

```
result = openai.images.generate(model=IMAGE_MODEL, prompt=prompt,
size="1024x1024")
b64 = result.data[0].b64_json
```

Embed that base64 string directly in an `<img>` tag as a *data URI* and return the page as `text/html` (template is in the solution):

```

```

The older DALL·E 3 model returned a temporary URL; the `gpt-image-*` models always return base64. See the [image generation guide](#).

Test: `/gpt/image?prompt=A+cat+astronaut+on+the+moon`

Exercise 9 — POST route (POST `/gpt/ask`)

Real AI APIs use **POST**: the prompt travels in the JSON request **body**, not the URL. This is cleaner, more secure, and handles large prompts. Flask reads it with `request.get_json()`; read `body["prompt"]`.

Test (curl):

```
curl -X POST http://localhost:3000/gpt/ask \
-H "Content-Type: application/json" \
-d '{"prompt": "What is the capital of France?"}'
```

Or in Postman: **POST** `http://localhost:3000/gpt/ask`, Body → raw → JSON → `{ "prompt": "What is the capital of France?" }`.

Exercise 10 — Multi-turn chat (POST `/gpt/chat`)

This is how every chatbot actually works. **The model has no memory — it is stateless.** The client keeps the full `messages` array and re-sends it every turn. The server passes it straight to GPT-4o, appends the reply, and returns the updated array. The client stores it and sends it back next turn.

Turn 1:

```
POST http://localhost:3000/gpt/chat
{ "messages": [{ "role": "user", "content": "What is the capital of France?" }] }
```

Turn 2 — paste the `messages` from the Turn 1 response, then add a follow-up:

```
{
  "messages": [
    { "role": "user",      "content": "What is the capital of France?" },
    { "role": "assistant", "content": "The capital of France is Paris." },
    { "role": "user",      "content": "What is its population?" }
  ]
}
```

GPT answers the second question with context from the first — because **you** sent the history.

Route summary

Method	Route	What it does
GET	<code>/ping</code>	Returns <code>pong</code> (no AI)
GET	<code>/tokens?prompt=...</code>	Count tokens without an API call
GET	<code>/ask?prompt=...</code>	Send a prompt to GPT-4o
GET	<code>/temperature?prompt=...&temp=...</code>	Call GPT-4o with a specific temperature
GET	<code>/gpt/reasoning?prompt=...&effort=...</code>	Call a reasoning model (Responses API)
GET	<code>/claude?prompt=...</code>	Send a prompt to Claude
GET	<code>/gpt/html?subject=...</code>	Generate an HTML page with GPT-4o
GET	<code>/claude/html?subject=...</code>	Generate an HTML page with Claude
GET	<code>/gpt/image?prompt=...</code>	Generate an image with gpt-image-2
POST	<code>/gpt/ask</code>	Send a prompt to GPT-4o via POST body
POST	<code>/gpt/chat</code>	Multi-turn conversation with GPT-4o

Official documentation

- [OpenAI API Reference](#)
- [Anthropic / Claude Docs](#)
- [Flask Quickstart](#)